Final Project Presentation-Quoridor

Group 5

B11901097 許庭愷

B11901053 徐啟惇

B11901132 林軒霆

動機、

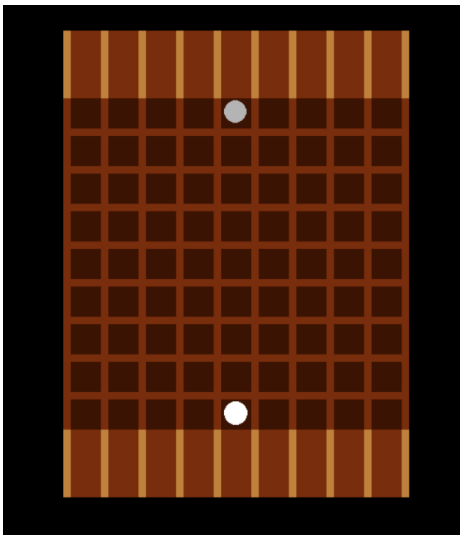
Quoridor 又名步步為營，是一款抽象策略桌遊，在玩家努力抵達終點的同時也要想辦法利用手中的木板阻饒對手的前進。由於它是十分經典的桌遊，我們便想將其寫成電腦遊戲，讓人能方便的體驗它所帶來的無窮樂趣。

分工表、

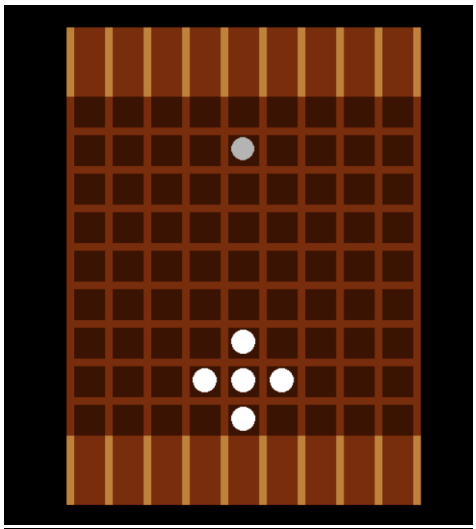
姓名	工作內容	比例
徐啟惇	大部分遊戲程式、report、校訂 report	1/3
許庭愷	遊戲程式、音效處理、report	1/3
林軒霆	遊戲程式、美工、簡報製作	1/3

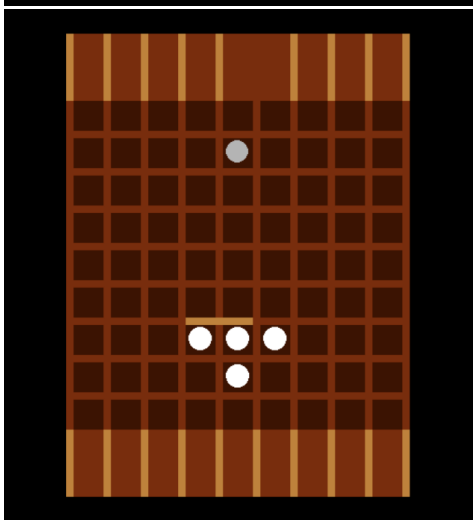
遊戲規則

1. 遊戲目標：率先將棋子移動至對手底線的玩家獲勝。
2. 遊戲設置：每位玩家有一個棋子，十個圍牆。遊戲開始時的設置如下圖所示。

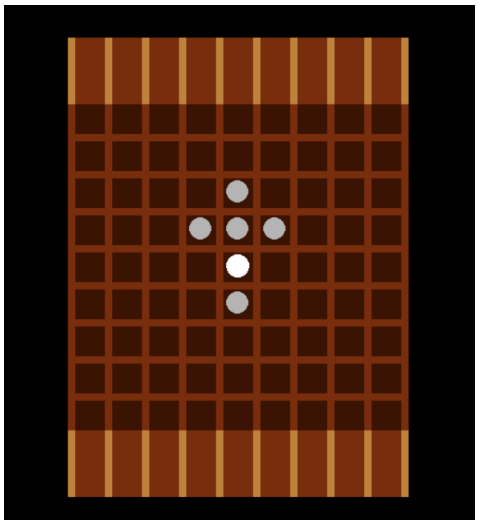


3. 遊戲流程：輪到其中一位玩家的回合時，以下兩個動作選擇一個來執行：
    - 1.移動棋子：移動棋子走一格，可以自由選擇前後左右方向， 但不可以移動到斜對角的格子，也不可以穿越圍牆，如下圖所示：

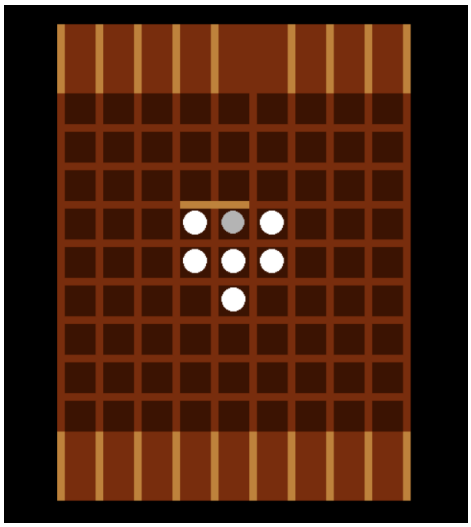




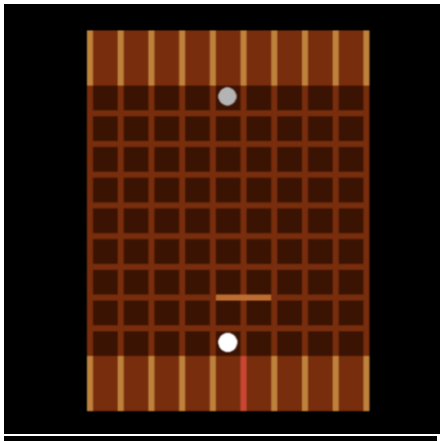
當與對手棋子面對面時，可以向跳棋一樣跳到對手棋子的正後方，如下圖所示：

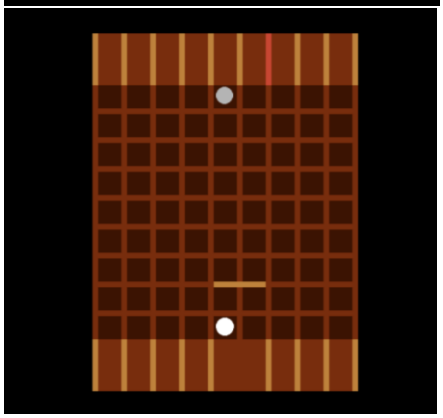


若對手棋子後方有圍牆阻擋時，可以跳至對手棋子的左方或右方，如下圖所示：



- 2.放置圍牆：放置一面圍牆來阻擋對手的前進，不能移動已放置的圍牆，圍牆一旦放完就只能移動棋子且所放置的圍牆不能使對手無法移動至他的終點(換句話說，只能使其繞遠路而行)。如下圖所示(第二張是放置完牆壁之後，玩家欄位裡面會失去一個圍牆)：





使用到的 library、

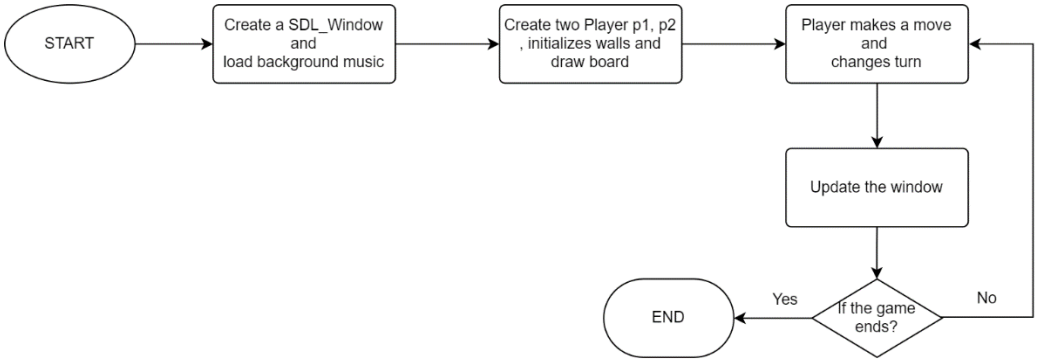
1. SDL.h：用來製作遊戲視窗以及介面。
2. SDL_image.h：用來處理.png 圖片。
3. SDL_mixer.h：用來處理背景音樂以及音效。

程式執行範例、

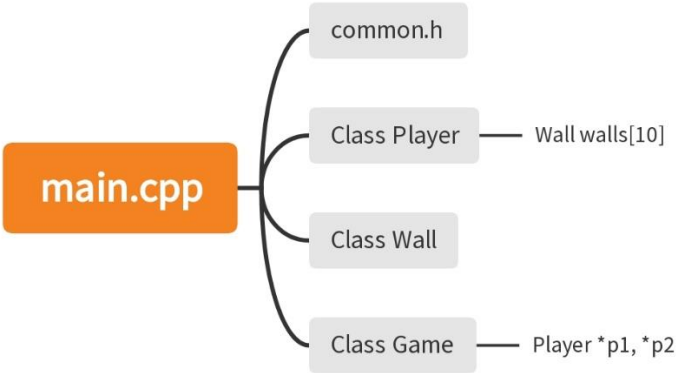
https://drive.google.com/file/d/164NG7izy3KS-VxRVks-cV5F22lKOcfCY/view?usp=share_link

程式流程圖與 OOP 程式架構、

1. 程式流程圖：



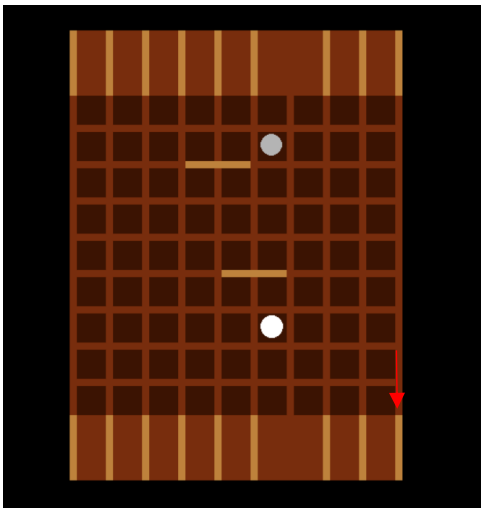
2. OOP 程式架構：
   我們將遊戲的一些要素拆分成幾個 C Header 檔，例如我們獨立寫一個圍牆的類別，裡面有圍牆的參數如座標、水平或垂直、是否使用過。再寫一個 Player 的類別，裡面有 Wall 類別的成員陣列等等。



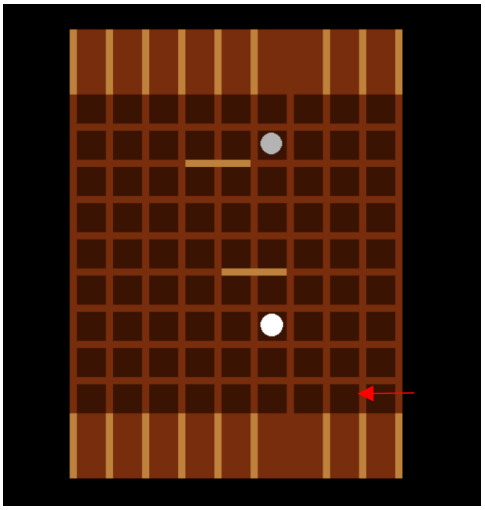
- common.h：包含一些 SDL 的功能以及一些共有的參數，讀取圖片的函式、載入音效的函式、格子大小的常數等等。
- Class Wall：包含圍牆所需的參數如顏色、座標、垂直與否。還有一些函式如改變圍牆的位置等。
- Class Player：玩家類別。大部分函式是關於棋子的移動。
- Class Game：控制遊戲的執行，依照不同事件去設計各物件的顯示以及物件間的互動。

common.h

1. SCRX：遊戲視窗的寬度。
2. SCRY：遊戲視窗的長度。
3. const int margins：橫向的一格寬度，如下圖箭頭所指。



4. const int squaresize：一個小正方形方塊的邊長，如下圖箭頭所指。



5. SDL_Window *screen
6. SDL_Renderer *renderer
7. unsigned int *pixptr
8. Mix_Music *backgroundMusic：背景音樂的指標。
9. Mix_Chunk *WallEffect：牆壁音效的指標。
10. Mix_Chunk *PawnEffect：棋子音效的指標。
11. inline void putpixel(int x, int y, int color)：輸入色塊位置以及顏色，放置一格色塊在視窗上。
12. inline int RGBtoInt(int r, int g, int b)：輸入顏色的參數，畫出顏色。
13. inline void draw_linex(int x, int ly, int width, int color)：輸入直線參數畫出直線。
14. inline void draw_square(int x, int y, int width, int height, int color1, int color2)：利用 draw_linex 畫出方形區域。
15. SDL_Surface *load_image(std::string filename)：輸入特定位置，讀取該位置上的 png 圖片。

16. void loadMusic()：初始化背景音樂*backgroundMusic、牆壁音效*WallEffect、以及棋子音效*PawnEffect。

Wall 類別、

1. int spx, spy：圍牆的座標。
2. bool placed：true 代表圍牆已經被放在遊戲版上不能移動，false 代表圍牆尚未被放置。
3. bool vertical：true 代表圍牆是垂直放置的，false 代表圍牆是水平放置的。
4. void set_wall(int spx, int spy)：遊戲開始時牆壁會先初始化，尚未被放置(placed = false)且是垂直在欄位中的(vertical = true)。
5. void set_focus()：當用滑鼠點擊時，產生不同顏色的圍牆以顯示被點擊。
6. void draw_wall(int color)：依據此時圍牆的參數以及顏色參數畫出牆壁(因為遊戲中有分成滑鼠選到的牆壁以及未選到的牆壁兩種顏色)。
7. void changepos(int spx, int spy, bool vercital): 在原本圍牆的位置畫上棋盤背景色彩好讓它看起來隱形，改變圍牆的位置，並將 placed = ture 代表圍牆已被使用過，並在放置牆壁時發出音效。最後再呼叫 draw_wall()，在目標的位置畫上圍牆。

Player 類別、

1. SDL_Surface *playerlook：棋子的圖片 。
2. SDL_Surface *playerlooksel：棋子影子(代表棋子可以走的格子)的圖片 。
3. SDL_Rect entrypoint：存取棋子在屏幕上的實際座標，與 locx,locy 的區別在於前者是在視窗中的絕對座標而後者是利用棋盤方格定義出的相對座標。
4. Wall walls[10] ：每位玩家有十個圍牆。
5. int desty：玩家獲勝終點的位置。
6. int locx, locy：棋子的座標。
7. int wselected：-1 代表目前狀態是什麼都沒有選取，0-9 則代表玩家選取的物件是圍牆，10 則代表玩家選取的是棋子。
8. bool mouse_over：true 代表滑鼠選取，false 代表滑鼠未選取。
9. void reset_player()：將 player 的 data member 初始化。
10. Player(int sy)：player 建構子，sy 表示玩家編號。
11. ~Player()：Player 的解構子，釋放多餘的空間。
12. bool can_up(Player *enemy)：判斷玩家的棋子是否可以往上。
13. bool can_left(Player *enemy)：判斷玩家的棋子是否可以往左。
14. bool can_down(Player *enemy)：判斷玩家的棋子是否可以往下。
15. bool can_right(Player *enemy)：判斷玩家的棋子是否可以往右。
16. void select(Player *enemy)：利用變數 wselected 控制選取圍牆、棋子、以及其他方格時分別在棋盤上產生棋子的影子、牆的影子、刪除已產生的棋子的影子。
17. void make_around(Player *enemy)：在所有可能移動的位置放置一個假的棋子(棋子的影子)，值得注意的是，在判斷棋子相鄰時可以走的位置時，可以判斷跳過棋子後有無遇到牆壁來區分狀況，若無的話直接跳過即可，有牆壁的話可以用敵方的 can_上下左右函數來判斷可以跳的位置。void del_around(Player *enemy)：把 17.中，所產生的棋子影子刪除。

18. void draw_player()：讀取並匯出棋子的圖像。
19. bool onplayer(int x, int y, int bx, int by)：判斷是否點擊到玩家的棋子。
20. bool onsquare(int x, int y, int bx, int by)：判斷是否點擊到某一個方格。
21. int possible_move(int x, int y, Player *enemy)：判斷可以走的格子，不行的話回傳 0，可以的話呼叫並回傳 move(int addx, int addy)的函式。
22. int move(int addx, int addy)：執行棋子移動的動作，再來播放棋子移動的音效並畫出新的棋子，最後判斷遊戲是否結束(也就是棋子是否抵達終點)。

Game 類別、

1. Wall demo：在放置圍牆時用來顯示所放置圍牆位置的物件。
2. bool wall_canplace(int spx, int spy, bool vertical)：判定當前所選取的圍牆是否可以放置，也就是是否有與現存在棋盤上的圍牆交叉。
3. void change_wall_place(int mx, int my, Wall *swall)：藉助 Wall 類別的 changepos 函數來將選取的圍牆擺設到目的地(demo 所在的位置)。
4. void preview_wall_place(int mx, int my)：當滑鼠移動過可擺放的位置，顯示一個顏色較淺的牆壁。
5. bool wall_place(int mx, int my, int *spx, int *spy, bool *vertical)：判斷滑鼠移過的位置是否在用來放置牆壁的縫隙中。
6. void change_turn()：換另外一位玩家執行他的回合。
7. Player *p1, *p2：玩家一的指標與玩家二的指標。
8. bool turn：true 代表現在是玩家一的回合，false 則是玩家二的回合。
9. bool game_end：true 代表遊戲結束，false 代表遊戲尚未結束。
10. Game()：Game 的建構子，賦予 p1 跟 p2 空間，設置玩家初始的圍牆，畫出遊戲板，並在玩家的欄位畫出圍牆。
11. void reset_game_status()：重新設置遊戲，game_end = false, turn = true。
12. void new_game()：開啟新的遊戲。
13. void reset_players()：重新設置玩家。
14. void set_walls()：重新設置牆壁。
15. Player *get_p1()：得到 p1 的指標。
16. Player *get_p2()：得到 p2 的指標。
17. void draw_board()：畫出遊戲板。
18. int which_wall(int x, bool turn)：利用滑鼠點擊的位置判斷所選取的圍牆是編號 0-9 中的哪一個。並回傳該圍牆的編號，如果沒有選取到圍牆回傳-1。
19. void move_event(int x, int y)：判斷滑鼠鼠標的位置以及動作。
20. void click_event(int button, int x, int y)：滑鼠按下去的動作，判斷滑鼠點到什麼(圍牆還是棋子)並執行後續動作。

main.cpp、

進行了各物件的呼叫以及初始化，並利用 while 迴圈以及 SDL_PollEvent()來讀取當前的 SDL_Event，並根據不同事件做出相對應的操作。(我們主要偵測的事件種類為: SDL_MOUSEBUTTONUP, SDL_MOUSEMOTION 以及 SDL_QUIT)。

參考資料、